

# Programming Assignments

---

## Assignments 1 – 4

|                     |                                                   |
|---------------------|---------------------------------------------------|
| <b>Assignment 1</b> | Linear Search & Binary Search                     |
| <b>Assignment 2</b> | Merge Sort, Quick Sort & Min/Max (D&C)            |
| <b>Assignment 3</b> | Matrix Chain Multiplication, Knapsack & Kruskal's |
| <b>Assignment 4</b> | Floyd-Warshall, TSP & Dijkstra's Algorithm        |

# Assignment - 1

Write a C Program to implement Linear Search and Binary Search with Algorithm

## 1. Linear Search

### Algorithm:

1. Start from index 0
2. Compare each element with the key
3. If match, return index
4. If end reached, element not found

### Code:

```
#include <stdio.h>

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5, key = 30;
    int result = linearSearch(arr, n, key);
    if (result != -1)
        printf("Element found at index %d", result);
    else
        printf("Element not found");
    return 0;
}
```

### Output:

Element found at index 2

## 2. Binary Search

### Algorithm:

1. Set low = 0, high = n-1
2. Find mid = (low + high) / 2
3. If key == arr[mid], return mid
4. If key < arr[mid], search left half
5. If key > arr[mid], search right half
6. Repeat until low > high

### Code:

```
#include <stdio.h>

int main() {
    int n, key, low, high, mid;
    int arr[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```

printf("Enter sorted elements:\n");
for (int i = 0; i < n; i++)
scanf("%d", &arr[i]);

printf("Enter element to search: ");
scanf("%d", &key);

low = 0; high = n - 1;

while (low <= high) {
mid = (low + high) / 2;
if (arr[mid] == key) {
printf("Element found at position %d", mid + 1);
return 0;
} else if (arr[mid] < key)
low = mid + 1;
else
high = mid - 1;
}
printf("Element not found\n");
return 0;
}

```

#### ***Output:***

```

Enter number of elements: 5
Enter sorted elements:
10 20 30 40 50
Enter element to search: 30
Element found at position 3

```

#### ***Complexity Summary***

| Algorithm     | Best Case | Average  | Worst Case | Sorted? |
|---------------|-----------|----------|------------|---------|
| Linear Search | O(1)      | O(n)     | O(n)       | No      |
| Binary Search | O(1)      | O(log n) | O(log n)   | Yes     |

## Assignment - 2

Write a C Program to implement Merge Sort, Quick Sort and find Min/Max using Divide and Conquer

### 1. Merge Sort

#### Algorithm:

1. If array has 1 element, return
2. Divide array into two halves
3. Recursively sort left half
4. Recursively sort right half
5. Merge both sorted halves

#### Code:

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];

    for (i = 0; i < n1; i++) L[i] = arr[left + i];
    for (j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    i = 0; j = 0; k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7;
    printf("Before Sorting: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    mergeSort(arr, 0, n - 1);
    printf("\nAfter Sorting: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    return 0;
}
```

#### Output:

Before Sorting: 38 27 43 3 9 82 10  
After Sorting: 3 9 10 27 38 43 82

### 2. Quick Sort

#### Algorithm:

1. Choose last element as pivot
2. Partition: elements less than pivot go left, greater go right
3. Recursively apply to left and right subarrays
4. Repeat until array is sorted

#### **Code:**

```
#include <stdio.h>

void swap(int *a, int *b) { int temp = *a; *a = *b; *b = temp; }

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = 6;
    printf("Before Sorting: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    quickSort(arr, 0, n - 1);
    printf("\nAfter Sorting: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    return 0;
}
```

#### **Output:**

```
Before Sorting: 10 7 8 9 1 5
After Sorting: 1 5 7 8 9 10
```

### 3. Min & Max using Divide and Conquer

#### **Algorithm:**

1. If array has 1 element, it is both min and max
2. If array has 2 elements, compare and return min & max
3. Divide array into two halves
4. Recursively find min & max of each half
5. Compare results and return overall min & max

#### **Code:**

```
#include <stdio.h>

void findMinMax(int arr[], int low, int high, int *min, int *max) {
    if (low == high) { *min = *max = arr[low]; return; }
    if (high == low + 1) {
        if (arr[low] < arr[high]) { *min = arr[low]; *max = arr[high]; }
        else { *min = arr[high]; *max = arr[low]; }
        return;
    }
}
```

```

int mid = (low + high) / 2;
int lMin, lMax, rMin, rMax;
findMinMax(arr, low, mid, &lMin, &lMax);
findMinMax(arr, mid + 1, high, &rMin, &rMax);
*min = (lMin < rMin) ? lMin : rMin;
*max = (lMax > rMax) ? lMax : rMax;
}

int main() {
int arr[] = {3, 5, 1, 8, 2, 9, 4, 7};
int n = 8, min, max;
findMinMax(arr, 0, n - 1, &min, &max);
printf("Array: ");
for (int i = 0; i < n; i++) printf("%d ", arr[i]);
printf("\nMinimum: %d\nMaximum: %d", min, max);
return 0;
}

```

### ***Output:***

```

Array: 3 5 1 8 2 9 4 7
Minimum: 1
Maximum: 9

```

### ***Complexity Summary***

| Algorithm   | Best Case     | Average       | Worst Case    |
|-------------|---------------|---------------|---------------|
| Merge Sort  | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ |
| Quick Sort  | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      |
| Min/Max D&C | $O(n)$        | $O(n)$        | $O(n)$        |

## Assignment - 3

Write a C Program to implement Matrix Chain Multiplication, 0/1 Knapsack Problem and Kruskal's Algorithm with Algorithm

### 1. Matrix Chain Multiplication

#### Algorithm:

1. Take n matrices and their dimensions
2. Use DP table to store minimum multiplications
3. For each chain length, try all possible split points
4. Find split point giving minimum cost
5. Store result in table and print minimum multiplications

#### Code:

```
#include <stdio.h>

int matrixChain(int p[], int n) {
    int dp[n][n];
    int i, j, k, l, q;
    for (i = 1; i < n; i++) dp[i][i] = 0;
    for (l = 2; l < n; l++) {
        for (i = 1; i < n - l + 1; i++) {
            j = i + l - 1;
            dp[i][j] = 99999;
            for (k = i; k < j; k++) {
                q = dp[i][k] + dp[k+1][j] + p[i-1] * p[k] * p[j];
                if (q < dp[i][j]) dp[i][j] = q;
            }
        }
    }
    return dp[1][n-1];
}

int main() {
    int p[] = {10, 30, 5, 60};
    int n = sizeof(p) / sizeof(p[0]);
    printf("Matrix Dimensions: ");
    for (int i = 0; i < n; i++) printf("%d ", p[i]);
    printf("\nMinimum number of multiplications: %d", matrixChain(p, n));
    return 0;
}
```

#### Output:

```
Matrix Dimensions: 10 30 5 60
Minimum number of multiplications: 4500
```

### 2. 0/1 Knapsack Problem

#### Algorithm:

1. Take number of items, their weights and values
2. Take knapsack capacity W
3. Build DP table of size (n+1) x (W+1)
4. For each item, decide to include or exclude
5. If weight fits, take max of include or exclude
6. Answer is dp[n][W]

#### Code:

```

#include <stdio.h>

int max(int a, int b) { return (a > b) ? a : b; }

int knapsack(int W, int wt[], int val[], int n) {
    int dp[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i-1] <= w)
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}

int main() {
    int val[] = {60, 100, 120};
    int wt[] = {10, 20, 30};
    int W = 50, n = 3;
    printf("Values : 60 100 120\n");
    printf("Weights: 10 20 30\n");
    printf("Capacity: %d\n", W);
    printf("Maximum Profit: %d", knapsack(W, wt, val, n));
    return 0;
}

```

#### **Output:**

```

Values : 60 100 120
Weights: 10 20 30
Capacity: 50
Maximum Profit: 220

```

### **3. Kruskal's Algorithm (Minimum Spanning Tree)**

#### **Algorithm:**

1. Sort all edges by weight in ascending order
2. Initialize each vertex as its own parent (Union-Find)
3. Pick smallest edge
4. If it does not form a cycle, add it to MST
5. Use Union-Find to detect cycles
6. Repeat until MST has (V-1) edges

#### **Code:**

```

#include <stdio.h>
#include <stdlib.h>

struct Edge { int src, dest, weight; };
int parent[100], rnk[100];

int find(int i) {
    if (parent[i] != i) parent[i] = find(parent[i]);
    return parent[i];
}

void unionSet(int x, int y) {
    int px = find(x), py = find(y);
    if (rnk[px] < rnk[py]) parent[px] = py;
    else if (rnk[px] > rnk[py]) parent[py] = px;
    else { parent[py] = px; rnk[px]++; }
}

```



```

}

int compare(const void *a, const void *b) {
return ((struct Edge*)a)->weight - ((struct Edge*)b)->weight;
}

int main() {
int V = 4, E = 5;
struct Edge edges[] = {{0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}};
qsort(edges, E, sizeof(edges[0]), compare);
for (int i = 0; i < V; i++) { parent[i] = i; rnk[i] = 0; }

printf("Edges in MST:\nEdge\t\tWeight\n");
int totalWeight = 0, edgeCount = 0;
for (int i = 0; i < E && edgeCount < V - 1; i++) {
int x = find(edges[i].src), y = find(edges[i].dest);
if (x != y) {
printf("%d -- %d\t\t%d\n", edges[i].src, edges[i].dest, edges[i].weight);
totalWeight += edges[i].weight;
unionSet(x, y); edgeCount++;
}
}
printf("Total MST Weight: %d", totalWeight);
return 0;
}

```

#### **Output:**

```

Edges in MST:
Edge Weight
2 -- 3 4
0 -- 3 5
0 -- 1 10
Total MST Weight: 19

```

#### **Complexity Summary**

| Algorithm    | Time            | Space           | Technique           |
|--------------|-----------------|-----------------|---------------------|
| Matrix Chain | $O(n^3)$        | $O(n^2)$        | Dynamic Programming |
| 0/1 Knapsack | $O(n \times W)$ | $O(n \times W)$ | Dynamic Programming |
| Kruskal's    | $O(E \log E)$   | $O(V+E)$        | Greedy + Union-Find |

## Assignment - 4

Write a C Program to implement Floyd-Warshall, Traveling Salesman Problem and Dijkstra's Algorithm with Algorithm

### 1. Floyd-Warshall Algorithm (All Pair Shortest Path)

#### Algorithm:

1. Initialize distance matrix with given graph weights
2. Set distance 0 for same vertex, infinity if no edge
3. For each intermediate vertex k
4. For each pair (i,j) check if path through k is shorter
5. Update  $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$
6. Print final shortest distance matrix

#### Code:

```
#include <stdio.h>
#define INF 99999
#define V 4

void floydwarshall(int graph[V][V]) {
    int dist[V][V], i, j, k;
    for (i = 0; i < V; i++)
        for (j = 0; j < V; j++)
            dist[i][j] = graph[i][j];

    for (k = 0; k < V; k++)
        for (i = 0; i < V; i++)
            for (j = 0; j < V; j++)
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    printf("Shortest Distance Matrix:\n\t 0\t 1\t 2\t 3\n");
    for (i = 0; i < V; i++) {
        printf("%d\t", i);
        for (j = 0; j < V; j++) {
            if (dist[i][j] == INF) printf(" INF\t");
            else printf(" %d\t", dist[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int graph[V][V] = {
        {0, 3, INF, 7},
        {8, 0, 2, INF},
        {5, INF, 0, 1},
        {2, INF, INF, 0}
    };
    floydwarshall(graph);
    return 0;
}
```

#### Output:

```
Shortest Distance Matrix:
0 1 2 3
0 0 3 5 6
1 7 0 2 3
2 3 6 0 1
3 2 5 7 0
```

## 2. Traveling Salesman Problem (TSP)

### Algorithm:

1. Start from source vertex 0
2. Generate all permutations of remaining vertices
3. For each permutation, calculate total path cost
4. Include return path back to source
5. Track minimum cost among all permutations
6. Print minimum cost and optimal path

### Code:

```
#include <stdio.h>
#define V 4

int graph[V][V] = {
{0, 10, 15, 20}, {10, 0, 35, 25},
{15, 35, 0, 30}, {20, 25, 30, 0}
};
int visited[V], minCost = 99999;
int bestPath[V], currentPath[V];

void tsp(int curr, int count, int cost, int start) {
if (count == V && graph[curr][start] != 0) {
int total = cost + graph[curr][start];
if (total < minCost) {
minCost = total;
for (int i = 0; i < V; i++) bestPath[i] = currentPath[i];
}
return;
}
for (int i = 0; i < V; i++) {
if (!visited[i] && graph[curr][i] != 0) {
visited[i] = 1;
currentPath[count] = i;
tsp(i, count + 1, cost + graph[curr][i], start);
visited[i] = 0;
}
}
}

int main() {
for (int i = 0; i < V; i++) visited[i] = 0;
visited[0] = 1; currentPath[0] = 0;
tsp(0, 1, 0, 0);
printf("Optimal Path: 0 ");
for (int i = 1; i < V; i++) printf("-> %d ", bestPath[i]);
printf("-> 0\nMinimum Cost: %d", minCost);
return 0;
}
```

### Output:

```
Optimal Path: 0 -> 1 -> 3 -> 2 -> 0
Minimum Cost: 80
```

## 3. Dijkstra's Algorithm (Single Source Shortest Path)

### Algorithm:

1. Initialize distance of source as 0, all others as infinity
2. Mark all vertices as unvisited
3. Pick unvisited vertex with minimum distance
4. For each neighbor, calculate distance through current vertex

5. If new distance is smaller, update it
6. Mark current vertex as visited
7. Repeat until all vertices are visited

#### Code:

```
#include <stdio.h>
#define INF 99999
#define V 5

int minDistance(int dist[], int visited[]) {
    int min = INF, minIndex;
    for (int v = 0; v < V; v++)
        if (!visited[v] && dist[v] <= min) { min = dist[v]; minIndex = v; }
    return minIndex;
}

void dijkstra(int graph[V][V], int src) {
    int dist[V], visited[V];
    for (int i = 0; i < V; i++) { dist[i] = INF; visited[i] = 0; }
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        visited[u] = 1;
        for (int v = 0; v < V; v++)
            if (!visited[v] && graph[u][v] && dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }
    printf("Vertex\t\tDistance from Source %d\n", src);
    printf("-----\t\t-----\n");
    for (int i = 0; i < V; i++) printf(" %d\t\t\t%d\n", i, dist[i]);
}

int main() {
    int graph[V][V] = {
        {0, 4, 0, 0, 8}, {4, 0, 8, 0, 0},
        {0, 8, 0, 7, 0}, {0, 0, 7, 0, 9}, {8, 0, 0, 9, 0}
    };
    dijkstra(graph, 0);
    return 0;
}
```

#### Output:

```
Vertex Distance from Source 0
-----
0 0
1 4
2 12
3 19
4 8
```

#### Complexity Summary

| Algorithm      | Time     | Space    | Technique           |
|----------------|----------|----------|---------------------|
| Floyd-Warshall | $O(V^3)$ | $O(V^2)$ | Dynamic Programming |
| TSP            | $O(n!)$  | $O(n)$   | Backtracking        |
| Dijkstra       | $O(V^2)$ | $O(V)$   | Greedy              |